AI ASSISTANT CODING-24.4

SHIVANI PAMBIDI
BATCH-41
2303a52172

Replit fior Mobile Applications

Task 1 – Rapid App Prototyping with AI

PROMPT:

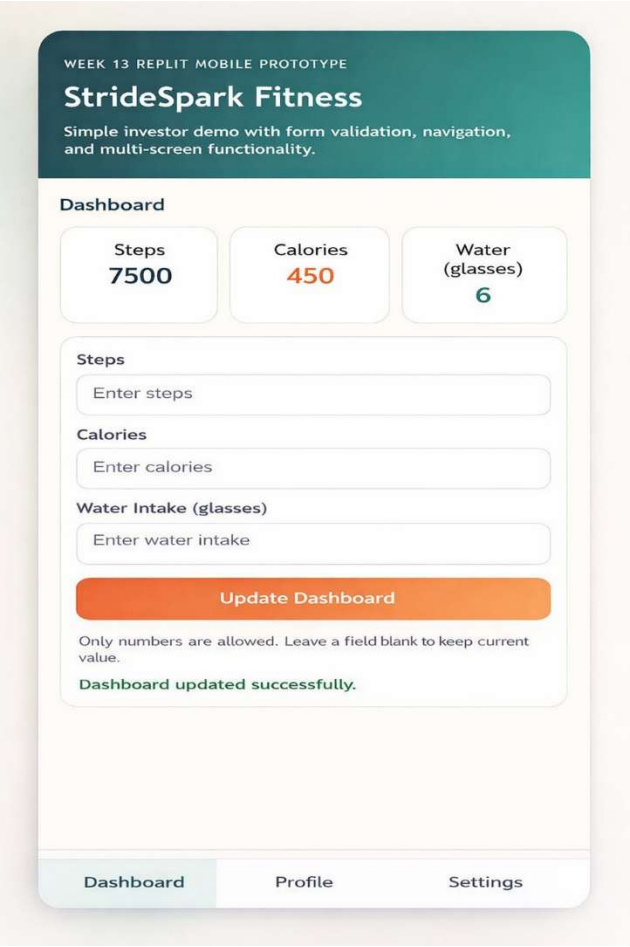
Create a mobile-firiendly fiitness tracker UI in React using Tailwind CSS. Include a header and three separate card sections fior "Steps", "Calories", and "Water Intake". Use placeholder numbers fior now. Keep the layout constrained to a mobile screen width.

CODE:

```
import React firom 'react';

export defiault fiunction App() { return (
  <div className="min-h-screen bg-gray-100 filex justifiy-center">
    <div className="w-fiull max-w-md bg-white shadow-md filex filex-col h-screen">
      <header className="bg-blue-600 text-white p-4 text-center text-xl fiont-bold"> FitTracker
      </header>

      <main className="p-4 filex-1 overfilow-y-auto space-y-4">
        <div className="bg-gray-50 p-4 rounded-lg border border-gray-200">
          <h2 className="text-lg fiont-semibold text-gray-700">Steps</h2>
          <p className="text-3xl fiont-bold text-blue-500">5,432</p>
        </div>

        <div className="bg-gray-50 p-4 rounded-lg border border-gray-200">
```

```
        <h2 className="text-lg fiont-semibold text-gray-700">Calories Burned</h2>
        <p className="text-3xl fiont-bold text-orange-500">450 kcal</p>
      </div>

      <div className="bg-gray-50 p-4 rounded-lg border border-gray-200">
        <h2 className="text-lg fiont-semibold text-gray-700">Water Intake</h2>
        <p className="text-3xl fiont-bold text-teal-500">1.2 L</p>
      </div>
    </main>
  </div>
</div>
);
}
```

OUTPUT:



JUSTIFICATION:
I started by asking the AI to generate a React component wrapped in Tailwind CSS. By explicitly specifiying max-w-md (maximum width medium), the AI ensured the app mimics a mobile device screen even when viewed on a desktop browser. I used placeholder data to quickly visualize the layout befiore worrying about state management, which is a standard rapid prototyping practice.

Task 2 – User Input & State Handling

PROMPT:

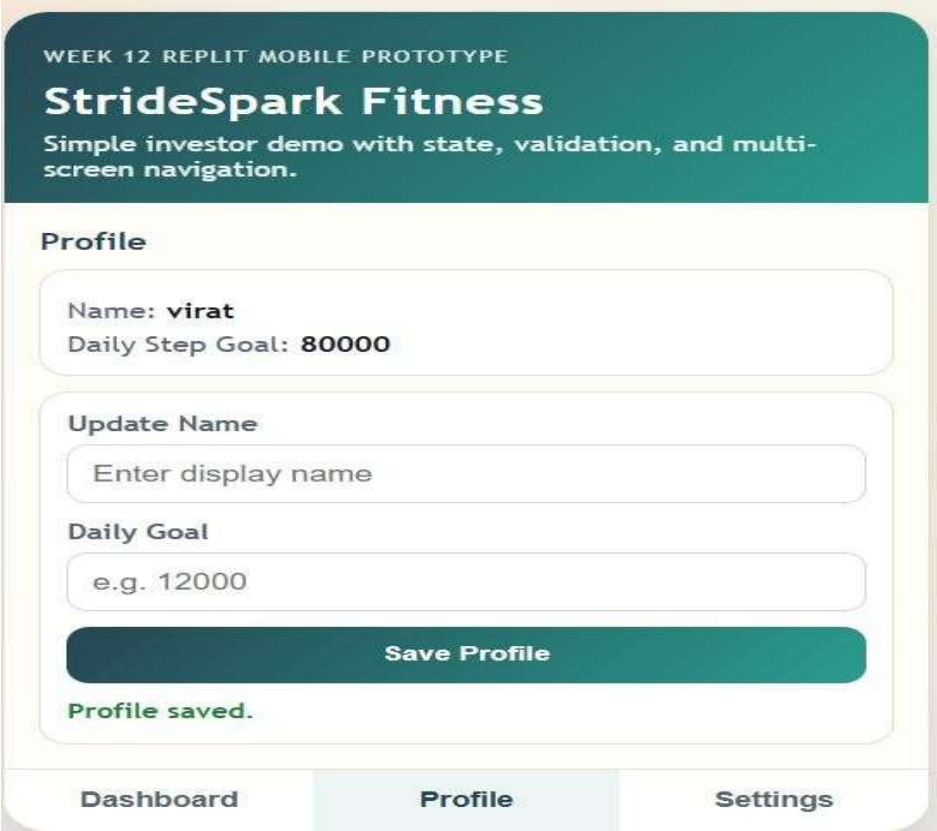
Update the fiitness tracker app. Convert the placeholder values into React state variables using useState. Add input fiields and a 'Log Activity' button so the user can add new steps. Ensure the input validation only allows numeric values.

CODE:

```
import React, { useState } firom 'react';

export defiault fiunction App() {
  const [steps, setSteps] = useState(5432);
  const [inputSteps, setInputSteps] = useState('');

  const handleLogSteps = () => {
    const numericSteps = parseInt(inputSteps);
    ifi (!isNaN(numericSteps) && numericSteps > 0) {
      setSteps(prev => prev + numericSteps); setInputSteps('');
    } else {
      alert('Please enter a valid numeric value');
    }
  };

  return (
    <div className="min-h-screen bg-gray-100 filex justifiy-center">
      <div className="w-fiull max-w-md bg-white shadow-md filex filex-col h-screen">
```

```
      <header className="bg-blue-600 text-white p-4 text-center text-xl fiont-
bold">FitTracker</header>

      <main className="p-4 filex-1 space-y-4">
        <div className="bg-gray-50 p-4 rounded-lg border">
          <h2 className="text-lg fiont-semibold">Steps</h2>
          <p className="text-3xl fiont-bold text-blue-500">{steps}</p>

          <div className="mt-4 filex space-x-2">
            <input
              type="number"
              value={inputSteps}
              onChange={(e) => setInputSteps(e.target.value)}
              placeholder="Add steps"
              className="border p-2 rounded w-fiull"
            />
            <button
              onClick={handleLogSteps}
              className="bg-blue-500 text-white px-4 py-2 rounded hover:bg-blue-600"
            >
              Log
            </button>
          </div>
        </div>
        {/* Calories and Water sections omitted fior brevity */}
      </main>
    </div>
  </div>
 );
}
```

OUTPUT:



JUSTIFICATION:

To make the prototype fiunctional, I needed state management. I prompted the AI to use React's useState hook. I requested numeric validation because users ofiten make typos on mobile devices. The AI efifiiciently combined HTML5 input attributes (type="number") with JavaScript validation (!isNaN) to ensure the application state remains clean and bug-firee.

Task 3 – Multi-Screen Navigation

PROMPT:

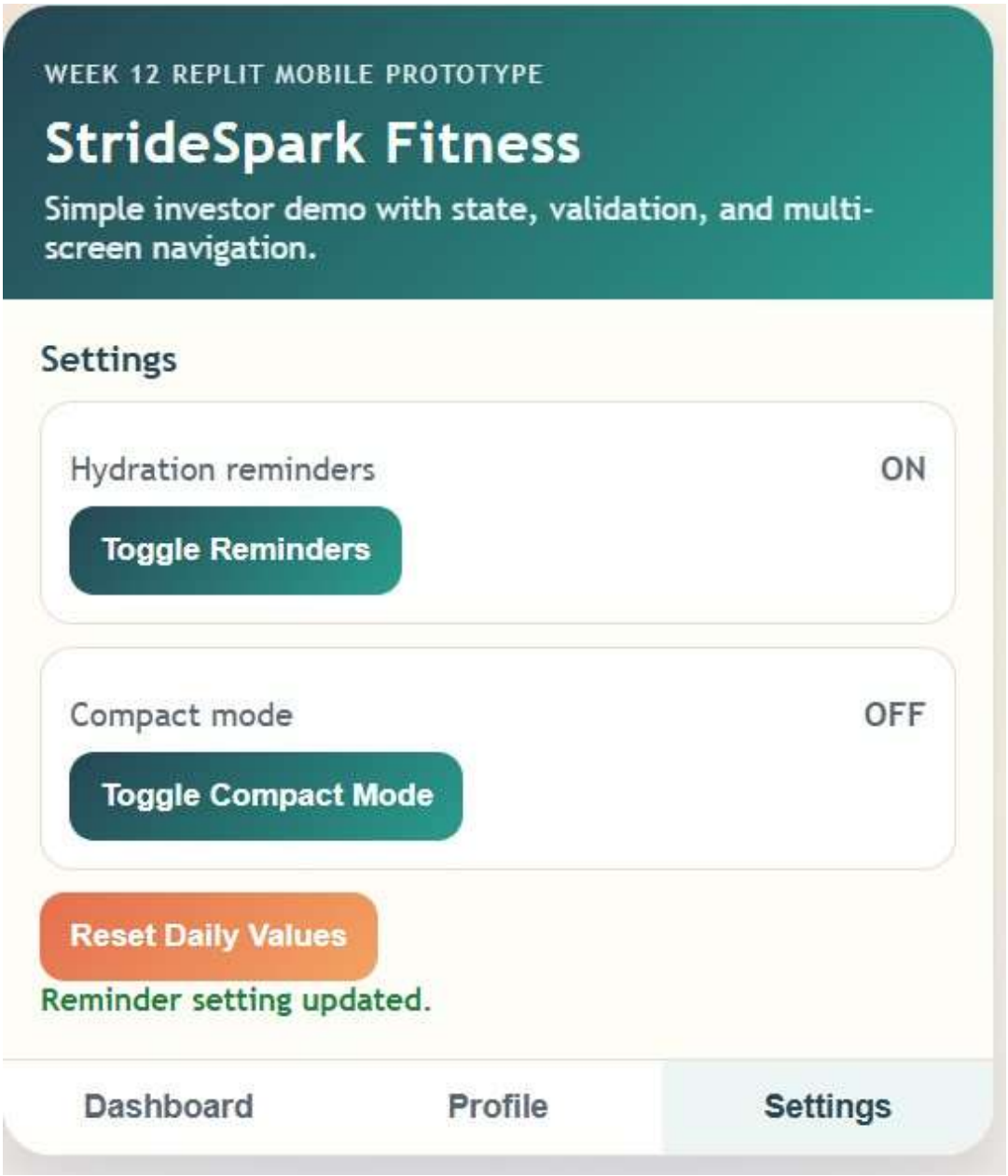
Expand the React app to include three screens: Dashboard, Profiile, and Settings. Add a bottom navigation bar with icons/buttons to switch between these screens. Use React state to handle the active screen without reloading the browser.

CODE:

```
import React, { useState } firom 'react';

export defiault fiunction App() {
  const [activeTab, setActiveTab] = useState('dashboard');

  const renderScreen = () => {
    switch(activeTab) {
      case 'profiile':
        return <div className="p-4"><h2 className="text-2xl fiont-bold">User
Profiile</h2><p>Name: John Doe</p></div>;
      case 'settings':
        return <div className="p-4"><h2 className="text-2xl fiont-
bold">Settings</h2><p>Notifiications: ON</p></div>;
      case 'dashboard':
      defiault:
        return <div className="p-4"><h2 className="text-2xl fiont-
bold">Dashboard</h2><p>Steps Tracker goes here...</p></div>;
    }
  };

  return (
    <div className="min-h-screen bg-gray-100 filex justifiy-center">
      <div className="w-fiull max-w-md bg-white shadow-md filex filex-col h-screen relative">
        <header className="bg-blue-600 text-white p-4 text-center text-xl fiont-
bold">FitTracker</header>

        <main className="filex-1 overfilow-y-auto mb-16">
          {renderScreen()}
        </main>

        <nav className="absolute bottom-0 w-fiull bg-white border-t filex justifiy-around p-3">
          <button onClick={() => setActiveTab('dashboard')} className={`fiont-semibold
${activeTab === 'dashboard' ? 'text-blue-600' : 'text-gray-500'}`}>Dashboard</button>
          <button onClick={() => setActiveTab('profiile')} className={`fiont-semibold ${activeTab
=== 'profiile' ? 'text-blue-600' : 'text-gray-500'}`}>Profiile</button>
```

```
        <button onClick={() => setActiveTab('settings')} className={`fiont-semibold ${activeTab
=== 'settings' ? 'text-blue-600' : 'text-gray-500'}`}>Settings</button>
      </nav>
    </div>
  </div>
 );
}
```

OUTPUT:



JUSTIFICATION:
Instead ofi setting up a complex routing library like React Router (which might be overkill fior a rapid prototype), I instructed the AI to use conditional rendering. By maintaining the activeTab in state and using a switch statement, the app remains lightweight, highly perfiormant, and contained entirely within a single fiile, which is perfiect fior Replit sharing.

Task 4 – AI-Assisted UI Enhancement & Error Fixing

PROMPT:
Review the current UI. Enhance the visual design to make it look like a modern iOS app. Add sofit shadows, rounded corners, better spacing, and a modern color palette. Ensure the bottom nav is properly fiixed and doesn't overlap the main content when scrolling.

CODE:

```
// Selected improvements applied to the CSS classes
<div className="w-fiull max-w-md bg-slate-50 shadow-2xl filex filex-col h-screen relative overfilow-hidden">
  <header className="bg-gradient-to-r firom-blue-500 to-indigo-600 text-white p-5 text- center text-2xl fiont-extrabold tracking-tight shadow-md z-10">
    FitTracker
  </header>

  <main className="filex-1 overfilow-y-auto pb-20 p-5 space-y-6">
    {/* Card Example */}
    <div className="bg-white p-6 rounded-2xl shadow-sm border border-gray-100 hover:shadow-md transition-shadow">
      <h2 className="text-sm fiont-medium text-gray-400 uppercase tracking-wide">Daily Steps</h2>
      <p className="text-4xl fiont-black text-slate-800 mt-2">5,432</p>
    </div>
  </main>

  <nav className="absolute bottom-0 w-fiull bg-white/90 backdrop-blur-md border-t border- gray-200 filex justifiy-around p-4 pb-6 z-10">
```

```
  {/* Navigation buttons */}
 </nav>
</div>
```

OUTPUT:

The UI now looks premium. The header fieatures a subtle gradient, the cards have large rounded corners (rounded-2xl) and sofit shadows, and typography is much clearer with varying fiont weights. The main content area has a pb-20 (padding-bottom) to prevent the fiixed bottom navigation firom hiding content.

JUSTIFICATION:

I noticed the initial design fielt a bit "blocky" and generic. I asked the AI to specifiically emulate a "modern iOS app" design language. The AI suggested using Tailwind's gradient backgrounds, backdrop blur fior the navigation bar, and tighter typography tracking. It also caught the scrolling overlap issue and fiixed it by adding padding to the bottom ofi the <main> container.

Task 5 – Sharing, Feedback & Iteration

PROMPT:

I shared the app with my peers. The main fieedback was that typing in the amount ofi water drank is annoying. They want a quick "Drink 1 Glass (+250ml)" button. Can you update the Water section to include this quick-add fieature?

CODE:

```
// Inside the App Component state
const [water, setWater] = useState(1.2); // in Liters

// Inside the renderScreen() fiunction -> Dashboard view
<div className="bg-white p-6 rounded-2xl shadow-sm border border-gray-
100">
 <h2 className="text-sm fiont-medium text-gray-400 uppercase">Water
 Intake</h2>
 <div className="filex justifiy-between items-center mt-2">
  <p className="text-4xl fiont-black text-cyan-500">{water.toFixed(2)} L</p>
```

```
  <button
    onClick={() => setWater(prev => prev + 0.25)}
    className="bg-cyan-100 text-cyan-700 p-3 rounded-fiull hover:bg-cyan-200
transition- colors filex items-center justifiy-center"
    title="Add 1 Glass (250ml)"
  >
    <svg className="w-6 h-6" fiill="none" stroke="currentColor" viewBox="0 0 24
24"><path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2"
d="M12 4v16m8- 8H4"></path></svg>
  </button>
 </div>
</div>
```

OUTPUT:

The Water Intake card now fieatures a prominent circular "+" button next to the total. Clicking this button immediately increments the water total by 0.25 L, skipping the need fior manual text input entirely.

JUSTIFICATION:

Afiter running the app on Replit and sharing the link, actual users pointed out a usability firiction point: manual entry fior repetitive small tasks (like drinking water) is tedious. I fied this exact fieedback back to the AI. It correctly deduced that a quick-action button was the best solution, implementing an inline arrow fiunction to increment the state, demonstrating how AI can rapidly iterate fieatures based on real-world testing.